

Week10

Task1:

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
int main()
{
    struct stat original, hard, symbolic;
    int fd;
    char buffer[100];
    ssize_t n;
    fd = open("original.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1){
        perror("open");
        return 1;
    }
    write(fd, "Hello Linux\n", 12);
    close(fd);
    stat("original.txt", &original);
    printf("Original File\n");
    printf("Inode Number : %lu\n", original.st_ino);
    printf("Links      : %lu\n\n", (unsigned long)original.st_nlink);
    if (link("original.txt", "hardlink.txt") == -1)
    {
        perror("link");
        return 1;
    }
    if (symlink("original.txt", "symlink.txt") == -1)
    {
        perror("symlink");
        return 1;
    }
    stat("original.txt", &original);
    stat("hardlink.txt", &hard);
    lstat("symlink.txt", &symbolic);
    printf("After Creating Links\n\n");
    printf("Original File\n");
    printf("Inode Number : %lu\n", original.st_ino);
    printf("Links      : %lu\n\n",
           (unsigned long)original.st_nlink);

    printf("      : %lu\n\n",
           (unsigned long)original.st_nlink);
    printf("Hard Link\n");
    printf("Inode Number : %lu\n", hard.st_ino);
    printf("Links      : %lu\n\n",
           (unsigned long)hard.st_nlink);
    printf("Symbolic Link\n");
    printf("Inode Number : %lu\n\n", symbolic.st_ino);
    if (original.st_ino == hard.st_ino)
        printf("Hard link has the SAME inode as original.\n");
    if (original.st_ino != symbolic.st_ino)
        printf("Symbolic link has a DIFFERENT inode.\n");
    n = readlink("symlink.txt", buffer, sizeof(buffer) - 1);
    if (n != -1)
    {
        buffer[n] = '\0';
        printf("Symbolic link points to: %s\n", buffer);
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./inode
Original File
Inode Number : 65021
Links       : 1

After Creating Links

Original File
Inode Number : 65021
Links       : 2

Hard Link
Inode Number : 65021
Links       : 2

Symbolic Link
Inode Number : 65031

Hard link has the SAME inode as original.
Symbolic link has a DIFFERENT inode.
Symbolic link points to: original.txt
root@DESKTOP-VMN932K:~#
```

Task2:

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#define SIZE 4096
void traditional_io(const char *filename){
    int fd;
    char buffer[SIZE];
    ssize_t bytes;
    clock_t start, end;
    fd = open(filename, O_RDWR);
    if (fd == -1){
        perror("open");
        return;
    }
    start = clock();
    lseek(fd, 0, SEEK_SET);
    while ((bytes = read(fd, buffer, SIZE)) > 0){
        lseek(fd, -bytes, SEEK_CUR);
        write(fd, buffer, bytes);
        lseek(fd, 0, SEEK_CUR);
    }
    end = clock();
    printf("Traditional read/write time: %.6f seconds\n",
        (double)(end - start) / CLOCKS_PER_SEC);
    close(fd);
}
void memory_mapped_io(const char *filename){
    int fd;
    struct stat st;
    char *mapped;
    clock_t start, end;
    fd = open(filename, O_RDWR);
    if (fd == -1){
        perror("open");
        return;
    }

```

```

GNU nano 7.2
}
if (fstat(fd, &st) == -1)
{
    perror("fstat");
    close(fd);
    return;
}
if (st.st_size == 0){
    printf("File is empty.\n");
    close(fd);
    return;
}
start = clock();
mapped = mmap(NULL, st.st_size,
              PROT_READ | PROT_WRITE,
              MAP_SHARED, fd, 0);

if (mapped == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return;
}
printf("\nData using mmap():\n");
write(STDOUT_FILENO, mapped, st.st_size);
if (st.st_size >= 5)
{
    memcpy(mapped, "HELLO", 5);
}
msync(mapped, st.st_size, MS_SYNC);
end = clock();
printf("\n\nMemory-mapped I/O time: %.6f seconds\n",
      (double)(end - start) / CLOCKS_PER_SEC);
munmap(mapped, st.st_size);
close(fd);
}
int main()
{
    char filename[100];
    printf("Enter file name: ");

```

```

    char filename[100];
    printf("Enter file name: ");
    scanf("%99s", filename);
    printf("\n--- Traditional I/O ---\n");
    traditional_io(filename);
    printf("\n--- Memory-Mapped I/O ---\n");
    memory_mapped_io(filename);
    printf("\nComparison:\n");
    printf("Traditional I/O uses read() and write().\n");
    printf("Memory-mapped I/O uses mmap() and accesses file data through memory.\n");
    printf("mmap() can reduce the number of explicit read/write system calls.\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./mmap
Enter file name: input.txt

--- Traditional I/O ---
Traditional read/write time: 0.000040 seconds

--- Memory-Mapped I/O ---

Data using mmap():
Hello from input file

Memory-mapped I/O time: 0.000343 seconds

Comparison:
Traditional I/O uses read() and write().
Memory-mapped I/O uses mmap() and accesses file data through memory.
mmap() can reduce the number of explicit read/write system calls.
root@DESKTOP-VMN932K:~#

```